Homework 5: Ontology-based Semantic Grounding

AI Capstone 2026

Group 14

莊詔允 112550062　張予綸 112550066　陳柏宗 112550073

楊淯棋 112550076　邱倫恩 111550195

1 Introduction and Repository Contents

This report documents the semantic grounding ontology developed for Homework 5. The goal of the ontology is to represent task objects, task roles, and affordances in a way that supports reasoning over robot manipulation tasks. Instead of treating object labels as isolated strings, the ontology connects each observed object to its type, role, affordances, and inferred semantic classes.

The repository contains the following main files:

- `ontology/group-ontology.ttl`: group-authored ontology containing Group 14 instances, task extensions, and reasoning patterns.
- `ontology/inferred-results.ttl`: exported inferred graph after running the reasoner in Protégé.
- `ontology/imports/course-affordance.ttl`: provided course ontology used as the shared vocabulary.
- `queries/`: SPARQL query files for checking inferred classes.
- `results/`: saved query outputs and screenshots from Protégé.

The main task represented in the group project is cutlery arrangement, and the advanced task is a cookie stowage scenario where cookie boxes are assigned to drawers. The ontology still includes the required baseline objects for cup stacking, cutlery arrangement, and toy block collection.

2 Ontology Design and Rationale

The ontology separates four ideas that are easy to confuse: object type, task role, affordance, and instance. For example, `g14:fork01` is an individual object instance, `cap:Fork` is its object type, `g14:role_target` is its task role, and `cap:GraspingAffordance` is the affordance that allows it to be classified as graspable.

This separation is useful because not every task-relevant object should be treated as graspable. For example, a plate is important in the cutlery arrangement task, but it is modeled mainly as a reference and support object. Similarly, a basket or drawer can be a container target without being a direct grasp target.

2.1 Namespace Policy

Two namespaces are used:

- `cap:` `https://hcis.io/ontology/aicapstone/2026/`
  This namespace is reserved for the shared course vocabulary, including classes such as `cap:PhysicalObject`, `cap:GraspingAffordance`, `cap:Cup`, `cap:Knife`, and properties such as `cap:hasTaskRole` and `cap:hasPoseFrame`.
- `g14:` `https://hcis.io/ontology/aicapstone/2026/group14/`
  This namespace is used for Group 14-specific instances, roles, task extensions, and advanced classes such as `g14:Drawer`, `g14:CookieBox`, `g14:OpenableAffordance`, and `g14:assignedToContainer`.

2.2 Modeled Objects and Affordances

Object Instance	Object Type	Task Role	Affordance
g14:blueCup01, g14:pinkCup01	cap:Cup	g14:role_target	cap:GraspingAffordance, cap:StackabilityAffordance
g14:knife01, g14:fork01	cap:Knife, cap:Fork	g14:role_target	cap:GraspingAffordance
g14:plate01	cap:Plate	g14:role_reference	cap:SupportAffordance
g14:block01, g14:block02, g14:block03	cap:ToyBlock	g14:role_collectable	cap:GraspingAffordance
g14:basket01	cap:Basket	g14:role_container	cap:ContainmentAffordance
g14:table01	g14:Table	g14:role_reference	cap:SupportAffordance
g14:drawer01, g14:drawer02, g14:drawer03	g14:Drawer	g14:role_container	cap:ContainmentAffordance, g14:OpenableAffordance
g14:cookieBox01, g14:cookieBox02, g14:cookieBox03	g14:CookieBox	g14:role_target	cap:GraspingAffordance

Table 1: Modeled object instances, semantic types, roles, and affordances.

3 Advanced Task Extension

The advanced task is a cookie stowage task. In this scenario, a robot must identify cookie boxes and place them into assigned drawers. To represent this task, the ontology introduces three main extensions:

- `g14:CookieBox`: a physical object that can be grasped and moved.
- `g14:Drawer`: a container-like object that can receive cookie boxes and can be opened.
- `g14:assignedToContainer`: an object property linking each cookie box to its assigned drawer.

This extension keeps the same modeling style as the baseline tasks. Cookie boxes are modeled as target objects with grasping affordances, while drawers are modeled as container objects with containment and openable affordances.

4 Reasoning Patterns and Key Axioms

The ontology uses OWL class definitions to infer object categories from affordances. The most important inference is `cap:GraspableObject`. An object should be inferred as graspable when it is a physical object and has a grasping affordance.

The main inference patterns are:

1. **Graspable object**: inferred when a `cap:PhysicalObject` has some `cap:GraspingAffordance`.
2. **Stackable object**: inferred when a `cap:PhysicalObject` has some `cap:StackabilityAffordance`.
3. **Openable object**: inferred when a `cap:PhysicalObject` has some `g14:OpenableAffordance`.

These classes are defined with `owl:equivalentClass`, `owl:intersectionOf`, and `owl:someValuesFrom`. The purpose is to avoid manually asserting final inferred classes for every object. For example, `g14:blueCup01` is asserted as a `cap:Cup`; because cups have grasping and stackability affordances, the reasoner can classify it as both graspable and stackable. Likewise, drawers are inferred as openable through their openable affordance.

The inferred class memberships were generated in Protégé using the HermiT reasoner and exported to `ontology/inferred-results.ttl`.

5 SPARQL Query Results

The ontology was checked using SPARQL queries in Protégé through the Snap SPARQL Query view. The query outputs were saved as text files under `results/`, and the screenshots are included below.

5.1 Graspable Objects

The query `queries/graspable_objects.rq` retrieves individuals inferred as `cap:GraspableObject`. The expected results are the two cups, the knife and fork, three toy blocks, and three cookie boxes. Objects such as the plate, basket, table, and drawers are excluded because they are not modeled as direct grasp targets.

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX g14: <https://hcis.io/ontology/aicapstone/2026/group14/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a cap:GraspableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

Execute

?obj	?label	?role
g14:block01	toy_block_01^^xsd:string	g14:role_collectable
g14:block02	toy_block_02^^xsd:string	g14:role_collectable
g14:block03	toy_block_03^^xsd:string	g14:role_collectable
g14:blueCup01	blue_cup^^xsd:string	g14:role_target
g14:cookieBox01	cookie_box_1^^xsd:string	g14:role_target
g14:cookieBox02	cookie_box_2^^xsd:string	g14:role_target
g14:cookieBox03	cookie_box_3^^xsd:string	g14:role_target
g14:fork01	cutlery_fork^^xsd:string	g14:role_target
g14:knife01	cutlery_knife^^xsd:string	g14:role_target
g14:pinkCup01	pink_cup^^xsd:string	g14:role_target

Figure 1: SPARQL result for inferred graspable objects.

5.2 Openable Objects

The query `queries/openable_objects.rq` retrieves individuals inferred as `g14:OpenableObject`. The result contains the three drawer instances, which are modeled with `g14:OpenableAffordance`.

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX g14: <https://hcis.io/ontology/aicapstone/2026/group14/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label
WHERE {
  ?obj a g14:OpenableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
}
ORDER BY ?obj
```

Execute

?obj	?label
g14:drawer01	drawer_1^^xsd:string
g14:drawer02	drawer_2^^xsd:string
g14:drawer03	drawer_3^^xsd:string

Figure 2: SPARQL result for inferred openable objects.

5.3 Stackable Objects

The query `queries/stackable_objects.rq` retrieves individuals inferred as `g14:StackableObject`. The result contains the two cup instances, since cups are modeled with a stackability affordance.

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX g14: <https://hcis.io/ontology/aicapstone/2026/group14/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label
WHERE {
  ?obj a g14:StackableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
}
```

Execute

?obj	?label
g14:pinkCup01	pink_cup^^xsd:string
g14:blueCup01	blue_cup^^xsd:string

Figure 3: SPARQL result for inferred stackable objects.

6 Design Choices and Limitations

Several modeling choices were made to keep the ontology clear and task-oriented.

First, the baseline toy block task uses three block instances instead of one. This better represents a small collection task and gives the query results more than one collectable object.

Second, the plate, basket, table, and drawers are not included in the graspable result. This is intentional. These objects are task-relevant, but they play reference, support, containment, or opening roles rather than direct grasp-target roles in our current modeling.

Third, the advanced cookie stowage task uses `g14:assignedToContainer` to represent the intended drawer assignment for each cookie box. This makes the advanced task more specific than simply saying that cookie boxes are graspable.

One limitation is that the ontology is mostly static. It can represent object types, roles, and affordances, but it does not directly model changes over time, such as a drawer becoming open after a successful pulling action. Another limitation is that OWL reasoning explains semantic classifications, but it does not replace geometric checks such as collision, reachability, or actual grasp success in simulation.

7 Future Work

A useful next step would be to add SHACL validation. For example, SHACL could check whether every task object has a pose frame, whether every target object has an object label, or whether every cookie box is assigned to exactly one drawer. This would complement the current OWL reasoning layer: OWL is used for inference, while SHACL would be used for validating whether the graph satisfies expected task constraints.

Another possible extension is to connect the ontology more directly with the Physical AI pipeline by linking ontology individuals to Isaac Sim object names, pose frames, and policy evaluation results.

8 Conclusion

The ontology provides a compact semantic grounding layer for the course manipulation tasks. It reuses the shared course vocabulary for baseline concepts, adds Group 14-specific extensions for the cookie stowage task, and uses OWL reasoning to infer graspable, stackable, and openable objects. The resulting inferred graph and SPARQL outputs make the modeled object knowledge inspectable and easier to connect with the robot task pipeline.